I — Foundations and long-term research (cognitive architectures, world models, developmental AI, experience-based learning)

0. Role in the system

This section is background for the whole design and the direct reference set for the P6 track (test-time / experience-based learning). It answers two questions. First: what would a future "thinking core" that learns the world from its own experience need in order to replace the language-model reader at the centre of the assistant? The Thousand Brains Project, I-JEPA / V-JEPA 2, the Alberta Plan and Lake et al. give four concrete answers (reference-frame sensorimotor modules with voting; predictive objectives in representation space; a base agent of perception + policies + value functions + transition model learned continually; causal, compositional model building with learning-to-learn). Second: what is the cognitive-architecture view of memory, impasses and chunking that the Skill tier and the Notebook imitate? The Standard Model of the Mind (Laird, Lebiere, Rosenbloom 2017) gives the component list, the cognitive cycle and the three learning mechanisms (procedural composition/chunking, reinforcement tuning of utilities, declarative learning). BabyLM shows how to budget and score sample-efficient learning. After reading this pack an agent must be able to (a) write the Standard-Model-to-module mapping table into the design document and keep the Notebook / Skills / gate / cascade / sleep modules consistent with it, (b) implement the small reference algorithms in §2.1 (evidence-based hypothesis update with voting, JEPA representation-space prediction step, CEM planning against a learned predictor, cognitive-cycle agent loop with chunking), and (c) set up a BabyLM-style budgeted evaluation for any future experience-learning core.

1. Papers

1.1 The Thousand Brains Project: A New Paradigm for Sensorimotor Intelligence (Clay et al., 2024) — arXiv:2412.18354

Problem and claim. Deep learning works on static datasets but struggles in diverse, open real-world settings. The paper proposes an alternative modelled on the neocortex: a "thousand-brains" system built from many identical, repeating **learning modules (LMs)** modelled on cortical columns. Each LM learns complete object models through **sensorimotor interaction**, stores them in **reference frames**, estimates and produces movement, and communicates with other LMs over a shared **Cortical Messaging Protocol (CMP)**. Learning is described as fast and associative, with no sharp boundary between learning and inference. **Monty** is the project's first (deliberately simple) implementation.

Headline results. The fetched paper text contains no accuracy tables; Section 5 only describes the experimental setup (Habitat simulator; one object per episode; YCB dataset of 77 objects such as cup, bowl, chain, hammer; two sensors: a 10x-zoomed patch for learning and an unzoomed "view finder" used at episode start; time discretised as *step* = one action + one observation, *episode* = one object until a terminal condition, *epoch* = one episode per object; also 2-D agents on Omniglot images and RGBD photos). The project's benchmark page (secondary source, not the paper — `https://docs.thousandbrains.org/docs/benchmark-experiments`) reports: YCB 10-object suite accuracy 87.00% (`randomrot_rawnoise_10distinctobj_surf_agent`) to 100.00%; YCB 77-object suite 92.21% (`randrot_noise_77obj_5lms_dist_agent`, 5 LMs, rotation error 49.39°) to 100.00% (`base_77obj_surf_agent`, rotation error 3.44°, 17 min); unsupervised learning first-epoch 40–80%, later epochs 80–98.89%; real-world iPad images 25.00% (multi-object) to 70.83%; COWS compositional sets 2.33–39.85% ("not currently expected to have good performance"). `[details from secondary — verify in current docs]`

Method — equations. No equations appear in the fetched text; the paper states the evidence rules in words (Sections 9.10–9.12, 10.1–10.2), which §2.2 formalises as *design*. Stated ranges: feature-match term in $[0, 1]$ (added only, proportional to the feature difference below a per-feature tolerance, 1 for a perfect match, 0 at or above tolerance); morphology (pose) term in $[-1, 1]$ (-1 if no stored point lies within the search radius of the predicted location; otherwise the angle between sensed and stored point normals maps from $+1$ at 0° to -1 at 180°, with 90° as the limit for the principal-curvature direction); total per-step update in $[-1, 2]$; vote evidence rescaled to $[-1, 1]$ per sending LM; received votes combined as a distance-weighted average in $[-1, 1]$ and added to the hypothesis evidence.

Method — algorithm. *Learning module structure (Sec. 3.3, 8–10).* 1. **Input**: features at poses. A feature is a feature ID (from a sensor module) or an object ID (from a lower-level LM). Every input carries the pose relative to the body: x, y, z location plus three orthonormal vectors (rotation; from the point normal and the two principal-curvature directions at the patch centre). 2. **CMP message** fields: location; 3×3 orthonormal morphological vectors; non-morphological features (colour, curvature; optional); confidence in $[0, 1]$; a boolean "use this message"; sender ID; sender type. Models themselves are never shared over CMP. 3. **Buffer** (short-term memory): observations of the current episode only; reset each episode; supplies the previous location for displacement computation. 4. **Graph memory** (long-term): one graph per object in 3-D Cartesian space, object-centric reference frame; node = pose + list of features; edges store displacements. A new point is not added if it is too similar to an existing one "by some threshold" (value not given), so storage concentrates where features change quickly. New observations are rotated/translated into the object frame using the detected pose. 5. **Learning phases** (9.1): matching phase (identify object + pose), then a specified number of exploratory steps, then store/update the model; unsupervised (no labels); models updated after every episode in training mode. 6. **Evidence-based LM** (9.9–9.12): hypothesis = (object, location, rotation) with a continuous

evidence value. Initialise with all objects and all stored locations; two rotations per location (normal + curvature direction; 180° ambiguity), or N sampled rotations about the normal when the principal curvatures are equal (N not given). Evidence starts at 0. Each step: rotate the sensed displacement by each hypothesis's rotation, add to its location to get a search location, find nearest stored nodes within the search radius, score with the pose term and the feature term above, add to accumulated evidence, and carry the search location forward as the hypothesis's new location. Objects are updated independently (parallelisable). 7. **Outputs** (Sec. 10, each step): (i) most likely object ID + pose = argmax evidence over objects and poses; (ii) a vote, sent laterally, only if the LM received input this step; (iii) a goal state (suggested action), decomposable by another LM or translated to motor commands. Every LM has a direct motor output. 8. **Possible matches** (10.1): a user-set percentage p (example 20%); a hypothesis is "possible" if its evidence exceeds $\max - p \cdot \max$. Hypotheses with zero/negative evidence are not possible; if every object is non-positive the LM creates a new model. 9. **Voting** (10.2): votes contain the set of possible objects and poses (not one hypothesis), evidence scaled to $[-1, 1]$; optionally only higher-evidence votes are sent; votes are transformed by the relative displacement between the sensors (assumes shared reference frames; otherwise a learned transform is needed); the receiver adds a distance-weighted average of nearby votes. Votes carry no modality-specific features, so LMs vote across modalities; enables "flash inference" without movement. 10. **Terminal conditions** (10.3): per LM *match*, *no match* (new graph created) or *time out* (max steps); the episode ends once a parameter-defined minimum number of LMs reach a terminal state (parameter name/value not in fetched text). 11. **Heterarchy** (10.4): bottom-up = lower LM's object ID + pose becomes a feature in the higher LM (tire → car; the car LM never sees the tire's 3-D structure); top-down = bias the lower LM's hypothesis space, and carry goal states; lateral = votes, possibly across levels. 12. **Action policies** (Sec. 11): *curvature-informed* (model-free; follow minimal/maximal principal curvature, with avoidance steps for revisited locations); *hypothesis-testing* (model-based; "graph mismatch": propose the point on the most-likely object farthest from the second-most-likely graph, e.g. the mug handle vs. a soup can); *goal-state generator* (LM sends a goal state in body-centric coordinates; Euclidean distance is the current heuristic).

Hyperparameters and setup. Monty = sensor modules + learning modules + motor system + environment, all connected via CMP; any component is replaceable if it follows the interface. YCB 77 objects; patch sensor zoom 10x; the only numeric thresholds in the text are the 20% possible-match example and the $[0, 1]$ confidence range. Core principles (Sec. 2.2): sensorimotor learning and inference; modular structure (one algorithm for all modalities); CMP; voting; reference frames; rapid continual learning; model-free and model-based policies. Capabilities list (Sec. 2.4) has 18 items including flash inference, learning from "a small number of samples", continual learning "maintaining previously learned representations", occlusion, categories, compositional objects, deformations, scale, object states, goals, abstract concepts, language, theory of mind.

Ablations that matter. None in the fetched text (not in fetched text — verify in paper §5 and project docs).

Code / data / license. Code: `https://github.com/thousandbrainsproject/tbp.monty` (MIT license; experiment configs in `src/tbp/monty/conf/experiment/`; benchmarks in `benchmarks/`). Docs: `https://docs.thousandbrains.org/`; API: `https://api-monty.thousandbrains.org`. Partially funded by the Gates Foundation; described as early beta.

Stated limitations. Monty is "a first, simple implementation"; no physics (objects do not move); scale not varied; hierarchical/compositional representations and hierarchical model-based policies are future work; model-based avoidance, feature-space comparisons for higher-level LMs and RL-learned policies are future work; evidence subtraction by features "only halfway achieved"; votes need a learned transform when reference frames differ.

Use in our design. The P6 "thinking core" blueprint takes four things: (1) a *buffer vs. graph memory* split (episode-scoped scratch vs. long-term object models) that mirrors Notebook core block vs. Notebook store; (2) *evidence accumulation over hypotheses with a percentage band for "possible matches"* as the model for how the gate keeps several candidate plans alive and only commits when one dominates; (3) *voting across independent modules* as the pattern for the verifier cascade combining independent, modality-agnostic signals; (4) *goal states decomposed down a heterarchy* as the pattern for plan → sub-task → tool call. For a single-user local system: one LM-equivalent per repository "object" is overkill; start with one evidence table over candidate skills/plans. PDF: https://arxiv.org/pdf/2412.18354 — Code: https://github.com/thousandbrainsproject/tbp.monty

1.2 Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (Assran et al., 2023) — arXiv:2301.08243

Problem and claim. Invariance-based self-supervised methods need hand-crafted view augmentations (biases, hard to transfer across modalities); masked pixel-reconstruction methods (MAE) need little prior knowledge but learn lower-level features that require fine-tuning. **I-JEPA** predicts, from a single context block, the *representations* of several target blocks of the same image, with the loss in representation space. Two design choices give semantic features: large, semantic target blocks and a sparse but informative context block. No augmentations.

Headline results. ImageNet-1K linear probe top-1 (Table 1): ViT-B/16 72.9 (600 ep), ViT-L/16 77.5 (600 ep), ViT-H/14 79.3 (300 ep), ViT-H/16 at 448 px 81.1 (300 ep); vs. MAE 68.0 / 76.0 / 77.2 (1600 ep), CAE 70.4 / 78.1, data2vec ViT-L/16 77.3; augmentation methods DINO ViT-B/8 80.1, iBOT ViT-L/16 81.0, SimCLR v2 RN152 79.1. ImageNet 1% (Table 2): ViT-L/16 69.4, ViT-H/14 73.3, ViT-H/16-448 77.3; MAE 67.1 / 71.5; data2vec 73.3; MSN 75.7. Transfer linear probe (Table 3, ViT-H/14): CIFAR100 87.5, Places205 58.4, iNat18 47.6 (MAE 77.3 / 55.0 / 32.9; iBOT ViT-L/16 88.3 / 60.4 / 57.3). Low-level tasks (Table 4): Clevr/Count 86.7, Clevr/Dist 72.4 (MAE 90.5 / 72.4; DINO 86.6 / 53.4; iBOT 85.7 / 62.8). ImageNet-22K scaling (Table 5): ViT-H/14 CIFAR100 89.5, Places205 57.8, iNat18 50.5, Count 88.6, Dist 75.0; ViT-G/16 89.5 / 59.1 / 55.3 / 86.7 / 73.0. Compute: ViT-H/14 pretrains in under 1,200 GPU-hours (16 A100, under 72 h); >2.5x faster than iBOT ViT-S/16 and >10x more efficient than MAE ViT-H/14; ~7% slower per iteration than MAE but converges in ~5x fewer iterations.

Method — equations. (Sec. 3) Image split into N non-overlapping patches. Target encoder $f_{\bar{\theta}}$ gives $s_y = \{s_{y_1}, ..., s_{y_N}\}$; context encoder f_θ encodes the masked context to $s_x = \{s_{x_j}\}_{j \in B_x}$; predictor g_ϕ takes s_x plus one mask token per target patch $j \in B_i$ (shared learnable vector + positional embedding) and outputs $\hat{s}_y(i)$. Loss:

$$\mathcal{L} = \frac{1}{M} \sum_{i=1}^{M} \sum_{j \in B_i} \left\| \widehat{s}_{y_j}(i) - s_{y_j} \right\|_2^2$$

M = number of target blocks (4); B_i = patch indices of target block i; $\widehat{s}_{y_j}(i)$ = predicted representation of patch j for block i; s_{y_j} = target-encoder representation of patch j (targets are taken by masking the *output* of the target encoder, not its input). ϕ and θ are trained by gradient descent; $\bar{\theta}$ is an exponential moving average of θ (the EMA formula itself is not written in the fetched text; momentum schedule below). The paper states the EMA target encoder is essential for training with ViTs.

Method — algorithm. 1. Sample $M = 4$ target blocks, each with aspect ratio in $(0.75, 1.5)$ and scale in $(0.15, 0.2)$ of the image (may overlap). 2. Sample one context block with scale in $(0.85, 1.0)$ and unit aspect ratio; remove any patches overlapping the target blocks. 3. Run f_θ on the context patches only; run $f_{\bar{\theta}}$ on the full image; mask its output to the target blocks. 4. For each target block i (so M predictor passes), feed s_x and the mask tokens with positional embeddings for B_i to g_ϕ. 5. Compute the loss above; update θ, ϕ by AdamW; update $\bar{\theta} \leftarrow$ EMA of θ.

Hyperparameters and setup. (Appendix A and released config `configs/in1k_vith14_ep300.yaml`.) AdamW; default batch size 2048 (config: `batch_size: 128` per GPU, 16 A100-80G); lr linear warmup to 10^{-3} then cosine to 10^{-6} (paper: from 10^{-4} over 15 epochs; config: `start_lr 0.0002, warmup 40, final_lr 1.0e-06, ipe_scale 1.0`); weight decay linearly $0.04 \to 0.4$; EMA momentum $0.996 \to 1.0$ linearly; target encoder initialised identical to the context encoder; predictor embedding dim 384 (`pred_emb_dim: 384`), heads = backbone's, depth 6 (ViT-B/16), 12 (ViT-L/16, ViT-H/14, ViT-H/16), 16 (ViT-G/16); no [cls] token; bfloat16; crop 224 (`crop_scale [0.3, 1.0]`); masks: `aspect_ratio [0.75,1.5], enc_mask_scale [0.85,1.0], pred_mask_scale [0.15,0.2], num_enc_masks 1, num_pred_masks 4, min_keep 10, allow_overlap false, patch_size 14`; epochs 300 (ViT-H), 600 (ViT-B/L). Linear eval: frozen encoder, average-pooled patch features (last layer or concat of last 4), LARS, batch 16384, 50 epochs, step decay 10x every 15 epochs. 1% ImageNet "low-shot": fine-tuning 50 epochs, AdamW, cosine, batch 512, layer decay 0.75, label smoothing 0.1.

Ablations that matter. (1% ImageNet top-1.) Prediction space (ViT-L/16): target-encoder output 66.9 (500 ep) vs. pixels 40.7 (800 ep). Masking (ViT-B/16, 300 ep): multi-block 54.2 (context ratio 0.25); rasterised quadrant 15.5; block(0.6) 20.2; random(0.6) 17.6. Number of targets: 1 → 9.0, 2 → 22.0, 3 → 48.5, 4 → 54.2. Target scale (4 targets, context 0.85–1.0): (0.075,0.2) 19.2; (0.1,0.2) 39.2; (0.125,0.2) 42.4; (0.15,0.2) 54.2; (0.2,0.25) 38.9; (0.2,0.3) 33.6. Context scale: (0.40,1.0) 31.2; (0.65,1.0) 47.1; (0.75,1.0) 49.3; (0.85,1.0) 54.2. Predictor depth (ViT-L/16, 500 ep): 6 → 64.0, 12 → 66.9. Predictor width (ViT-L/16, 600 ep, fine-tune): 384 → 70.7, 1024 → 68.4.

Code / data / license. `https://github.com/facebookresearch/ijepa` (checkpoints for ViT-H/14 300 ep IN1K, ViT-H/16 448 px, ViT-H/14 IN22K, ViT-G/16 IN22K; license: "see the LICENSE file" — not stated in the fetched README). Data: ImageNet-1K, ImageNet-22K.

Stated limitations. Trails view-invariance methods on some semantic transfer (iNat18 47.6 vs. iBOT 57.3); MAE better on Clevr/Count (90.5 vs 86.7); ViT-G/16 with larger patches does not improve low-level tasks; pixel-space prediction degrades linear probing substantially.

Use in our design. The P6 core takes the *predict-in-representation-space with an EMA target* recipe as the only loss family that avoids both pixel/token reconstruction and hand-crafted augmentations: for a code world-model, "context block" = visible span of a file/trace, "target blocks" = masked spans whose *embeddings* (not tokens) are predicted. The ablation message to carry over: targets must be large and semantic (span-level, not token-level), context must be sparse but informative, and several targets per context (4) are needed. For a single 12 GB GPU, scale down to a ViT-S-class predictor (depth 6, width 384) over a frozen code encoder. PDF: https://arxiv.org/pdf/2301.08243 — Code: https://github.com/facebookresearch/ijepa

1.3 V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning (Assran et al., 2025) — arXiv:2506.09985

Problem and claim. Can a model learn to understand, predict and plan in the physical world mainly by observation? Claim: internet-scale self-supervised video pretraining plus a small amount of unlabeled robot interaction data yields "a world model capable of planning in the physical world". Two stages: (1) action-free pretraining of V-JEPA 2 on >1M hours of video; (2) action-conditioned post-training of **V-JEPA 2-AC** on <62 h of unlabeled Droid robot video, then zero-shot deployment on Franka arms in two labs with image goals, no rewards, no task-specific training.

Headline results. Understanding (frozen encoder, 4-layer attentive probe, top-1): V-JEPA 2 ViT-g (1B) SSv2 75.3, Diving-48 90.1, Jester 97.7, K400 86.6, COIN 90.7, IN1K 84.6, avg 87.5; ViT-g at 384 px: 77.3 / 90.2 / 97.8 / 87.3 / 91.1 / 85.1, avg 88.2 (DINOv2 1.1B avg 81.1; PEcoreG 1.9B 82.3; SigLIP2 1.2B 81.1; InternVideo2-1B 87.0; V-JEPA 1 ViT-H 85.2). Scaling ablation (Fig. 3, six-task avg): ViT-L/16 on 2M videos 84.2 → +data (22M) 85.2 → +ViT-g 86.7 → +longer training (252K it) 87.5 → +progressive resolution 88.2 (cumulative +4.0). Epic-Kitchens-100 action anticipation recall@5 (action): ViT-g-384 39.7 (verb 63.6, noun 57.1); ViT-g 38.0; ViT-H 36.5; ViT-L 32.7; prior best PlausiVL (8B) 27.6 (+44% relative); Video-LLaMA 26.0; InAViT 25.8. Video QA with Llama 3.1 8B: PerceptionTest 84.0 (SFT, test), MVP 44.5, TempCompass 76.9, TemporalBench 36.7, TOMATO 40.3, TVBench 60.6, MVBench 73.5 (PLM 8B: 82.7 / 39.7 / 72.7 / 28.3 / 33.2 / 63.5 / 77.1). Robot (10 trials/task, success %): Lab 1, V-JEPA 2-AC: reach 100, grasp cup 70, grasp box 30, reach-with-object cup 90 / box 80, pick-and-place cup 80 / box 80 (Octo: 100 / 20 / 0 / 20 / 70 / 20 / 10); Lab 2, V-JEPA 2-AC: 100 / 60 / 20 / 60 / 70 / 80 / 50 (Octo 100 / 10 / 0 / 10 / 70 / 10 / 10; Cosmos: reach 80, grasp cup 0, grasp box 20, pick-and-place 0 / 0). Planning time per action: ~16 s (V-JEPA 2-AC, 800 samples, 10 CEM iterations, horizon 1) vs. ~4 min (Cosmos, 80 samples, 10 iterations), both on one RTX 4090.

Method — equations. Pretraining (Sec. 2.1, Eq. 1):

$$\min_{\theta, \phi, \Delta_y} \left\| P_\phi(\Delta_y, E_\theta(x)) - \mathrm{sg}(E_{\bar{\theta}}(y)) \right\|_1$$

x = masked video (patches dropped); y = the full video; E_θ = encoder (ViT); P_ϕ = predictor (ViT); Δ_y = learnable mask tokens marking dropped-patch positions; sg = stop-gradient; $E_{\bar{\theta}}$ = target encoder with $\bar{\theta}$ an EMA of θ; L1 loss applied only to masked

patches. Post-training teacher forcing (Sec. 3.1, Eq. 2):

$$\mathcal{L}_{\mathrm{tf}}(\phi) = \frac{1}{T}\sum_{k=1}^{T} \left\| P_\phi((a_t, s_t, E(x_t))_{t \le k}) - E(x_{k+1}) \right\|_1$$

x_k = frame k of a 16-frame clip; E = frozen V-JEPA 2 encoder, $z_k := E(x_k)$ (a $16 \times 16 \times 1408$ feature map per frame); s_k = 7-D end-effector state (3 position, 3 extrinsic Euler angles, 1 gripper) relative to the base; a_k = 7-D action = change of end-effector state between k and $k+1$; $T = 15$. Rollout loss (Eq. 3):

$$\mathcal{L}_{\mathrm{rollout}}(\phi) = \left\| P_\phi(a_{1:T}, s_1, z_1) - z_{T+1} \right\|_1, \quad T = 2 \text{ in practice}$$

Total (Eq. 4): $L(\phi) = \mathcal{L}_{\mathrm{tf}}(\phi) + \mathcal{L}_{\mathrm{rollout}}(\phi)$. Planning (Sec. 3.2, Eq. 5):

$$E(\widehat{a}_{1:T}; z_k, s_k, z_g) := \left\| P(\widehat{a}_{1:T}; s_k, z_k) - z_g \right\|_1, \qquad (a_i^\star)_{i \in [T]} := \arg\min_{\widehat{a}_{1:T}} E(\widehat{a}_{1:T}; z_k, s_k, z_g)$$

z_g = encoding of the goal image; z_k, s_k = current frame encoding and proprioceptive state.

Method — algorithm. *Stage 1 (pretraining).* Mask-denoising in representation space on VideoMix22M (~22M samples: SSv2 168K / 168 h, weight 0.056; Kinetics 400/600/700 733K / 614 h, 0.188; HowTo100M 1.1M / 134K h, 0.318; YT-Temporal-1B 19M / 1.6M h, 0.188, retrieval-curated; ImageNet 1M images duplicated to 16-frame clips, 0.250). Encoder ViT-g (>1B), 3-D RoPE, tubelets $2 \times 16 \times 16$, multiblock masking as in V-JEPA 1, predictor "similar to a ViT-small". Schedule warmup–constant–decay: 12K warmup iterations at 16 frames / 256 px, 228K constant, 12K cooldown raising clips to 64 frames / 384 px with linear LR decay (252K total; ~8.4x less GPU time than full-resolution from scratch; full-res ViT-g would need "roughly 60 GPU-years"). *Stage 2 (post-training).* Freeze the encoder; train a ~300M-parameter block-causal transformer predictor (24 layers, 16 heads, 1024 hidden) on ~62 h of Droid Franka Panda video (~23k trajectories; clips >= 4 s, 256 px, 4 fps, 16 frames; random-resize-crop aspect (0.75, 1.35)); interleave actions, states and frame features in time; each step attends to current and past features/actions/states; loss = teacher forcing + 2-step rollout. *Inference (MPC loop).* 1. Encode current frame and goal image. 2. Sample action sequences from Gaussians (zero mean, unit variance). 3. Roll out the predictor, score with the energy above. 4. Refit mean/variance on the top-k trajectories; repeat 10 iterations with 800 samples. 5. Execute only the first action; replan. Pick-and-place uses three image goals: grasp sub-goal for 4 steps, placement sub-goal for 10 steps, final goal for 4 steps.

Hyperparameters and setup. From the released configs (code pointers; not all are in the paper text): pretraining `configs/train/vitg16/pretrain-256px-16f.yaml`: `vit_giant_xformers`, `pred_depth 12`, `pred_embed_dim 384`, `pred_num_heads 12`, `use_rope true`, masks = 8 blocks at spatial scale 0.15 + 2 blocks at 0.7 (temporal scale 1.0, aspect 0.75–1.5), batch 24 per GPU on 16 nodes x 8 GPUs, fps 4, 16 frames, crop 256, `lr 0.000525`, `start_lr 0.0001`, `warmup 40`, `weight_decay 0.04`, `ema 0.99925`, `loss_exp 1.0` (L1), bfloat16, `epochs 800`, `ipe 300`. Post-training `configs/train/vitg16/droid-256px-8f.yaml`: batch 8 per GPU on 4 x 8 GPUs, 8 frames, 4 fps, predictor depth 24 / dim 1024 / 16 heads, `is_frame_causal true`, `lr 0.000425`, `start_lr 0.000075`, `final_lr 0.0`, warmup 15 epochs, anneal 15, `epochs 315`, `ipe 300`, `auto_regressive_steps 2`, `loss_exp 1.0`, representations normalised. Released V-JEPA 2 checkpoints: ViT-L/16 300M, ViT-H/16 600M, ViT-g/16 1B (256 px) and ViT-g/16 1B at 384 px (Hugging Face `facebook/vjepa2-*`); V-JEPA 2-AC ViT-g checkpoint; later V-JEPA 2.1 checkpoints ViT-B 80M … ViT-G 2B at 384 px.

Ablations that matter. The four scaling ingredients above (+1.0, +1.5, +0.8, +0.7 points). Pick-and-place without sub-goals is not solved (limitation below).

Code / data / license. `https://github.com/facebookresearch/vjepa2` (code MIT, three dataset-utility files Apache-2.0; model weight license not stated in fetched README); blog `https://ai.meta.com/blog/v-jepa-2-world-model-benchmarks`. Data: VideoMix22M sources above; Droid.

Stated limitations. Camera-pose sensitivity: action axes inferred from a monocular uncalibrated view; the authors "manually tried different camera positions". Long-horizon planning limited by autoregressive error accumulation and exponential growth of the search space with horizon (predictions up to ~16 s ahead). Goals must be images; language goals are future work. EK100: 1-second anticipation only, closed vocabulary, kitchens only. Models scaled only to ~1B.

Use in our design. The two-stage recipe is the template for the P6 core: pretrain a representation predictor on passive data (repositories, traces, diffs), then post-train a small *action-conditioned* predictor (actions = edits/commands; "proprioception" = repo/test state) on a few dozen hours of the user's own sessions with the encoder frozen; plan by sampling action sequences and minimising distance to a goal embedding (goal = "tests pass" state embedding). Carry over: teacher forcing + short rollout loss, block-causal predictor, replanning after one action, sub-goals for long tasks, 300M-class predictor on a frozen encoder fits one consumer GPU. PDF: https://arxiv.org/pdf/2506.09985 — Code: https://github.com/facebookresearch/vjepa2

1.4 The Alberta Plan for AI Research (Sutton, Bowling, Pilarski, 2022) — arXiv:2208.11173

Problem and claim. A 5–10-year research plan to understand and build "long-lived computational agents that interact with a vastly more complex world" and learn to predict and control their sensory input, especially a scalar reward. The focus is basic understanding, not applications. Commitments: experience (observation, action, reward time series) is the only basis for learning; scalar reward defines the goal; no special training sets, human assistance or privileged access to the environment's internals; **temporal uniformity** ("all times are the same with respect to the algorithms running on the agent" — learning, planning and meta-learning run every step, no training phases); methods that scale with computation (the "bitter lesson"); reaction time traded against decision quality; multi-agent settings and intelligence amplification.

Headline results. None — a plan, not an experimental paper. Only quantitative statements: 5–10 years; 12 steps; four base-agent components; Step 8 has six listed components (a–f); Step 4 has four sub-steps; Step 12 has two versions.

Method — equations. None in the fetched text.

Method — algorithm. *Agent–environment interface (Fig. 1).* Agent sends actions; environment returns an observation (does not fully reveal state) and a scalar reward; observations + actions + rewards = experience; all learning grounded in these signals.

Base agent (Fig. 2; an extension of the "Common Model of the Intelligent Agent" shared by AI, psychology, control theory, neuroscience, economics). 1. **Perception** updates the *state*, a summary of past experience used by every other component (examples: belief-state updating; stacking four Atari frames). Called "perhaps the least well understood" component; learning perception is open. 2. **Reactive policies**: a primary policy trained to maximise reward plus other policies for other quantities. 3. **Value functions**: one per policy, used to learn it. 4. **Transition model**: learns dynamics from actions, rewards and states; predicts future states and rewards; planning imagines outcomes which the value functions evaluate. *The twelve steps.* 1. Representation I — continual supervised learning with given features: meta-learned per-feature step sizes, online feature normalisation, tracked bias (named: NADALINE, IDBD, Autostep, Adam, RMSprop, batch norm). 2. Representation II — supervised feature finding: build new features from existing ones under a fixed resource budget; discard weak features, replace by "smart generation" then test and rank. 3. Prediction I — continual GVF (generalized value function) prediction learning on non-i.i.d. real-time data, recurrent networks, moving to off-policy. 4. Control I — continual actor-critic: k-armed bandit → contextual bandit (softmax) → sequential with given features → sequential with feature finding. 5. Prediction II — average-reward GVF learning: differential value with subtracted average reward rate, or conventional value plus option duration. 6. Control II — continuing control problems: River Swim, Access-control Queuing, Jellybean World, GARNET; convert episodic Gym tasks to continuing. 7. Planning I — planning with average reward: incremental asynchronous dynamic programming, tabular first, then function approximation. 8. Prototype-AI I — one-step model-based RL with continual function approximation, no temporal abstraction: (a) recursive state update, (b) one-step model, (c) feature finding, (d) feature ranking, (e) model-to-ranking feedback, (f) search control (MCTS or prioritized sweeping). 9. Planning II — search control and exploration: asynchronous value iteration with function approximation, prioritized sweeping, small backups, MCTS to heuristic search. 10. Prototype-AI II — the STOMP progression (SubTask, Option, Model, Planning): top features become reward-respecting subtasks, each solved into an option with a learned option model; off-policy learning. 11. Prototype-AI III — Oak: feedback processes that assess and replace features, subtasks, options and option models; an "option keyboard" where chords of subtask components form combined options. 12. Prototype-IA — intelligence amplification: an agent boosting another agent's speed and decision-making, first an "exo-cerebellum", later an "exo-cortex", tested human–agent and agent–agent.

Hyperparameters and setup. None. Linear function approximation in Steps 1–2; recurrent networks with limited per-observation processing in Step 3; nonlinear continual function approximation from Step 8.

Ablations that matter. None.

Code / data / license. No code. (Not in fetched text — verify: later releases from the Alberta group are separate papers.)

Stated limitations. Later steps are "less concrete and likely to change"; no numeric benchmarks; learning perception unresolved.

Use in our design. The Alberta base agent is the cleanest statement of what the P6 core must eventually contain and what the current system lacks: today's assistant has *perception* (context assembly), a *reactive policy* (the worker) and a *value function* stand-in (verifier scores / gate calibration) but **no transition model**, so no planning by imagination — V-JEPA 2-AC (§1.3) is the candidate. Temporal uniformity is the principle behind "learn while working" (P6) as opposed to weekly sleep-time LoRA only; Steps 1–2 (per-feature step sizes, generate-and-test feature replacement under a budget) map directly onto how the Notebook and Skill library should admit, rank and retire entries continually. Step 12 (intelligence amplification) is literally the product: an agent amplifying one developer. PDF: https://arxiv.org/pdf/2208.11173

1.5 Building Machines That Learn and Think Like People (Lake et al., 2016) — arXiv:1604.00289

Problem and claim. Deep networks match or exceed humans on some tasks but differ in *what* they learn and *how*. Human-like machines must (i) build causal models of the world that support explanation and understanding rather than pattern recognition, (ii) ground learning in intuitive theories of physics and psychology, and (iii) use compositionality and learning-to-learn to generalise rapidly from little data. The authors frame it as "model building" vs. "pattern recognition" and argue for combining neural-network advances with structured cognitive models. Published in *Behavioral and Brain Sciences* 40, e253 (2017; online Nov 2016).

Headline results. Two challenge problems, with numbers from the text. *Characters challenge* (Sec. 3.1): MNIST (6,000 images per class, 60,000 training images) where convnets reach "below 1% test error", versus a person learning a new handwritten character from one example and then generating new examples, parsing it into parts, and creating new characters (Fig. 1); Bayesian Program Learning (BPL) performs one-shot classification "at human level performance" and outperforms convnets (Koch et al.); visual Turing test grids of 9 human vs 9 BPL samples (Fig. 5B, answer key "1,2;1,1"). (Omniglot size and the numeric BPL / human / convnet error rates are not in the fetched text — verify in paper §3.1/§4.2 and Lake et al. 2015.) *Frostbite challenge* (Sec. 3.2): DQN trained on 200 million frames per game, about 924 hours (~38 days) of experience, ~500x a professional gamer's ~2 hours, each frame replayed ~8 more times; with that experience DQN reached "less than 10% of human-level performance" in a controlled test; later variants 83% (DQN+) and 96% (DQN++) with far more experience; learning curve (Schaul et al.): ~46% after 231 h, 19% after 116 h, <3.5% after 2 h; random play ~1.5%; the authors reached expert-comparable scores after ~2 minutes of expert YouTube play and at most 15–20 minutes of playing. PhysNet (Lerer et al. 2016) needed 100,000–200,000 training scenes for one tower-stability task.

Method — equations. None in the fetched text (the paper is a position paper; BPL's generative model is described structurally, not with equations, in this text).

Method — algorithm. The core ingredients (Sec. 4), transcribed as the paper organises them. *4.1 Developmental start-up software* — early-present capacities, "intuitive theories" with causal structure resembling scientific theories. 1. **Intuitive physics** (4.1.1): infants expect objects to persist, be solid and cohesive; a physics-engine (simulation) approach predicts tower stability (Battaglia et al. 2013); contrasted with PhysNet's data needs. 2. **Intuitive psychology** (4.1.2): infants distinguish agents, expect goal-directed, efficient behaviour; generative models of action (Bayesian inverse planning) preferred over cue-based accounts; watching an expert can teach "birds are dangerous" without ever being attacked. *4.2 Learning as rapid model building* — learning

= constructing causal models that explain the data. 3. **Compositionality** (4.2.1): “new representations can be constructed through the combination of primitive elements”; structural descriptions of parts and relations; Frostbite scenes built from object types. In BPL a character is a simple stochastic program: primitives → sub-parts → parts → object template → exemplars, with relations between parts (figure labels “attached along”, “attached at start”); structure is shared through “the compositional reuse of stochastic primitives” and new characters are built by combining parts, sub-parts and relations. 4. **Causality** (4.2.2): causal models “represent hypothetical real world processes that produce the perceptual observations”; BPL treats concepts “as motor programs, or abstract causal descriptions of how to produce examples of the concept”; variational autoencoders sit at the non-causal end; neural networks can act as bottom-up proposers for structured inference. 5. **Learning-to-learn** (4.2.3): strong priors “must be making up the difference” when inference goes beyond the data; defined as the ways “learning a new task (or a new concept) can be accelerated through previous or parallel learning”; hierarchical Bayesian models (“a general prior on concepts is shared by multiple specific concepts”) and feature sharing in deep networks. (BPL lesion results per ingredient: not in fetched text — verify in paper §4.2.3.) *4.3 Thinking fast.* 6. **Approximate inference in structured models** (4.3.1): exact inference over program spaces is intractable; Monte Carlo sampling as a model of human inference, limited in vast hypothesis spaces; children’s “Aha!” moments suggest guided theory building; **amortized inference** compiles probabilistic inference into a feed-forward mapping, linking probabilistic models and deep learning. 7. **Model-based and model-free RL** (4.3.2): DQN is model-free (dopamine reward-prediction error); the brain also has a model-based system building a “cognitive map” for planning; model-based planning adapts to new goals when only the reward changes (modular environment model); habits form with routine use (shift from model-based to model-free control); plans can be cached into model-free values by simulated training data (Sutton 1990) during offline periods such as “dreaming or quiet wakefulness”; model-based behaviour becomes automatic with training (Economides et al. 2015). *Sec. 5 Responses to common questions* (partially fetched): humans’ richer prior experience justifies multi-task/transfer learning but networks may not generalise human-like without the right representational structure; cognitive plausibility (“software” before “hardware”) preferred over neural constraints for now; language builds on the core abilities present before children talk.

Hyperparameters and setup. Not applicable.

Ablations that matter. The Frostbite learning-curve numbers above are the paper’s sample-efficiency ablation of DQN against human learning time.

Code / data / license. No code in the paper. MIT DSpace copy under CC BY-NC-SA 4.0 (`http://hdl.handle.net/1721.1/112658`). Omniglot (Lake et al. 2015) is the dataset behind the Characters challenge.

Stated limitations. The paper’s “What this article is not” section and Sec. 5 concede that humans start with far richer experience; that architecture search is not yet algorithmic; that language is not treated. Sections 6 and the conclusions were beyond the fetched text (verify in paper §6).

Use in our design. The five ingredients are the acceptance criteria for any “thinking core” that is to replace the reader: it must represent *causal* models of code (what a change does, not what text follows), build them *compositionally* from reusable parts (skills), *learn to learn* (priors across tasks — the Notebook lessons and the LoRA archive are our explicit priors), and keep an *intuitive-physics-like* core model of the environment (types, control flow, filesystem, test semantics). Sec. 4.3.2 is the justification for the sleep module: cache verified model-based solutions (search traces) into model-free weights (LoRA) offline — “dreaming”. PDF: https://arxiv.org/pdf/1604.00289

1.6 Call for Papers — The BabyLM Challenge: Sample-efficient pretraining on a developmentally plausible corpus (Warstadt et al., 2023) — arXiv:2301.11796

Problem and claim. Children are exposed to “2M–7M words per year (Gilkerson et al., 2017)”, so a developmentally plausible pretraining budget up to age 12 is “between 24M–84M words”, orders of magnitude below modern LM pretraining (the paper cites Chinchilla and OPT but gives no token counts in text; Figure 1 visualises the scale). The shared task (with CoNLL and CMCL) asks for sample-efficient pretraining on a fixed, child-inspired corpus, to serve small-scale LM research, cognitive modelling of acquisition, low-resource NLP, and to “democratize research on pretraining”.

Headline results. The call itself reports none. From the Findings paper of the 2023 challenge (secondary source, arXiv:2504.08165 — `[details from secondary — verify]`): 31 papers, 162 models (Strict-Small 118 models / 29 participants; Strict 24 / 11; Loose 20 / 8). Aggregate scores: OPT-125M baseline Strict 0.60 (BLiMP 0.75, GLUE 0.70, MSGS 0.13, supplement 0.68), Strict-Small 0.50 (0.63 / 0.62 / 0.10 / 0.53); winner Strict: ELC-BERT (LTG-BERT-based) 0.74 (BLiMP 0.85, GLUE 0.78, MSGS 0.47, supplement 0.77); Strict-Small: ELC-BERT 0.66 (runner-up MLSM 0.61); Loose: Contextualizer 0.73 (MSGS 0.58). Skylines: Llama 2 70B 0.71; RoBERTa-base on its full corpus 0.70. Findings: LTG-BERT-style architecture gave most of the gain; curriculum learning (13 teams, 41.9%) mostly gave no consistent improvement; distillation (teacher–student, e.g. BabyLlama) often helped; data augmentation (Contextualizer) beat 40 epochs on the same samples; shorter inputs helped; Strict barely beat Strict-Small despite 10x data; Loose generally underperformed Strict-Small.

Method — equations. None in the call. Scoring (from the Findings paper, secondary): aggregate = weighted average with BLiMP + BLiMP-supplement 50%, (Super)GLUE 30%, MSGS 20%; MSGS reports a Matthews correlation where $+1$ = systematic linguistic generalisation and -1 = systematic surface generalisation.

Method — algorithm. *Tracks* (Sec. 3): **STRICT** — ~100M words (98.04M), only the provided dataset; **STRICT-SMALL** — ~10M words (9.96M), a roughly 10% uniform subsample, only the provided dataset; **LOOSE** — linguistic text capped at 100M words, but “unlimited non-linguistic data or text which differs from the restricted shared task” and other modalities welcome. *Dataset* (Table 1; words STRICT-SMALL / STRICT / share): CHILDES (child-directed speech) 0.44M / 4.21M / 5%; British National Corpus dialogue 0.86M / 8.16M / 8%; Children’s Book Test 0.57M / 5.55M / 6%; Children’s Stories Text Corpus 0.34M / 3.22M / 3%; Standardized Project Gutenberg Corpus 0.99M / 9.46M / 10%; OpenSubtitles 3.09M / 31.28M / 31%; QCRI Educational Domain Corpus (educational video subtitles) 1.04M / 10.24M / 11%; Wikipedia 0.99M / 10.08M / 10%; Simple Wikipedia 1.52M / 14.66M / 15%; Switchboard Dialog Act Corpus 0.12M / 1.18M / 1%; total 9.96M / 98.04M. *Evaluation* (Sec. 5): a shared pipeline “based in Google

Colab" with public code; "the pipeline assumes all models can be loaded and queried in HuggingFace's transformers library"; models must score sequences (log-likelihood or pseudo-log-likelihood) and be fine-tunable for classification; generation is not required; the abstract names "targeted syntactic evaluations and natural language understanding". *Baselines* (5.1): OPT (decoder-only), RoBERTa (encoder-only) and T5 (encoder–decoder) hyperparameters, trained from scratch on the fixed data, described as not strong. *Rules*: STRICT/STRICT-SMALL may not use pretrained models for anything (reranking, augmentation); no limits on hyperparameters or epochs; no compute cap, but estimated resources will be released and LOOSE may weigh efficiency. *Submission* (Sec. 6): model download link, a .zip of pipeline predictions, a short description, and (LOOSE) the extra data. *Timeline*: data Jan 2023; pipeline Mar 2023; results due 15 Jul 2023; papers 1 Aug 2023; presentation at CoNLL. Eval suite as actually run in 2023 (Findings, secondary): BLiMP (minimal pairs; correct if the acceptable sentence gets higher probability); BLiMP supplement (hidden: hypernyms, subject–auxiliary inversion, turn-taking, question–answer congruence, plus one more); (Super)GLUE: MRPC, QQP, SST-2, MNLI, QNLI, RTE, BoolQ, MultiRC, CoLA, WSC (fine-tuned); MSGS (fine-tune on confounded data, test on disambiguating data); optional age-of-acquisition surprisal correlation (7 teams).

Hyperparameters and setup. Word budgets 10M / 100M; proportions above; everything else left to participants.

Ablations that matter. From the Findings (secondary): Strict vs Strict-Small near-identical (data 10x does little at this scale); architecture > curriculum; distillation and augmentation help.

Code / data / license. `https://babylm.github.io/`; data and pipeline released by the organisers (licenses of the component corpora not in fetched text).

Stated limitations. Baselines weak by design; evaluation pipeline unspecified at call time; no compute cap; the age-12 budget is an approximation from 2–7M words/year.

Use in our design. BabyLM is the evaluation *protocol* for the P6 track: fix a word/token budget for anything the core is allowed to learn from (e.g. the user's own repositories and sessions, ~10M–100M tokens, which is exactly the scale of one developer's corpus), forbid pretrained-model leakage in the "strict" condition, and score with a fixed weighted suite (syntax-style minimal pairs for code, task fine-tuning, and an MSGS-style generalisation-vs-surface-heuristic test). The 2023 finding that architecture and distillation beat curricula tells us to spend effort on the learning mechanism, not on ordering the data. PDF: https://arxiv.org/pdf/2301.11796 — Site: https://babylm.github.io/

1.7 A Standard Model of the Mind: Toward a Common Computational Framework across Artificial Intelligence, Cognitive Science, Neuroscience, and Robotics (Laird, Lebiere, Rosenbloom, 2017) — AI Magazine 38(4), article 2744 (no arXiv id)

Problem and claim. Propose a community-consensus "standard model" of humanlike minds — computational entities whose structure and processing resemble human cognition — at the level of a *cognitive architecture*, synthesised from ACT-R, Soar and Sigma. Background: Newell's time-scale bands (biological, cognitive, rational, social; 12 orders of magnitude from ~100 µs to months); the model sits at the **deliberate act** level (~100 ms, a simple reaction), the "critical juncture" between neural processes and bounded rationality. A cognitive architecture is the fixed structure at that level; unlike a computer architecture its programs and data are limited to humanlike behaviour and are meant to be learned from experience except for a limited innate set. Open copy read: `https://scispace.com/pdf/a-standard-model-of-the-mind-toward-a-common-computational-f5mhxm3hk5.pdf` (AAAI OJS `https://ojs.aaai.org/index.php/aimagazine/article/view/2744` is bot-blocked; PDF link reported by aitopics: `https://aaai.org/ojs/index.php/aimagazine/article/download/2744/2671`). Pages 13–26 (not verified in fetched text).

Headline results. A consensus table, not experiments. Table 2 scores ACT-R, Soar and Sigma against the assumptions: by 2016 ACT-R and Soar agree in theory and differ only in how far perception/motor are implemented; Sigma partially disagrees on three cells — the "strong architectural distinction between procedural and declarative memories" and "the complete architectural nature of reinforcement learning"; 1993 versions of ACT-R/Soar disagreed or lacked theory mostly on perception/motor.

Method — equations. None.

Method — algorithm. *Components (Fig. 4).* **Working memory**; **declarative long-term memory**; **procedural long-term memory**; **perception** (possibly many modality modules, each with its own working-memory buffer); **motor** (possibly many, e.g. arm, leg). Each may decompose further (semantic vs episodic declarative memory; procedural matching/selection/execution stages; several working-memory buffers). *Connectivity.* Working memory is the global communication buffer. Perception, motor and declarative memory read/write only their own working-memory buffers. Procedural memory reads all of working memory but cannot access declarative memory directly, nor itself. Perception and motor may also connect directly. *Cognitive cycle (A4).* Each cycle (~50 ms in humans; the deliberate-act level ~100 ms): procedural memory matches rule-like conditions against working memory; metadata decides among matching rules; exactly **one deliberate act** is selected; its actions modify (possibly several items of) working memory; those changes may trigger further rule firings, declarative retrieval (cue placed in a designated buffer), motor actions, or top-down influence on perception, whose results return to working memory. Complex behaviour = a sequence of cycles (A5: no separate architectural module for global optimisation or planning; planning, language, theory of mind arise from sequences of primitive acts). "The restriction to selecting a single deliberate act per cycle yields a serial bottleneck in performance" while processing is parallel within and across modules (A3a/A3b). *Memory and content.* Symbols are the primitives over which relations are defined; structures = semantic networks, taxonomies; the model is agnostic on whether symbols are uninterpreted labels or distributed patterns; nonsymbolic processing for statistical/spatial reasoning. Numbers play two roles: explicit quantities, and **quantitative metadata** annotating symbols/relations to modulate processing — frequency, recency, co-occurrence, similarity, utility, activation. Metadata is strictly separate from domain data: relations cannot be defined over it and its types are fixed by the architecture. Working memory holds goals, intermediate results, task models and the retrieval/perception/motor buffers; no capacity number is given; constraints are the serial action bottleneck and an attentional bottleneck on perception. Procedural memory = pattern-directed rules with conditions over working memory and actions that change it. Declarative memory = persistent graph of symbolic relations with metadata, cue-based retrieval; one uniform memory vs separate semantic/episodic still debated. *Learning (C1–C5).* All long-term content (symbols and metadata) is learnable; "all

learning is incremental, and takes place online" as a side effect of performance; learning typically inverts the performance flow, running backward over internal representations of experience; longer-timescale learning builds on accumulated short-term learning and may include deliberation over past experiences. Three mechanism families: (1) **procedural composition** — new rules created by composing rule firings (ACT-R "rule composition" / production compilation; Soar **chunking**), yielding automatised behaviour (C3b); (2) **reinforcement learning** tuning the selection among competing deliberate acts via utilities (C3a); (3) **declarative learning** — creating new relations/facts and tuning their metadata such as activation, frequency, recency (C4). Perception and motor are assumed to learn too (D1b, D2b) though not fully implemented. (Impasses/substates are not described in this text.) *Table 1 assumptions.* A1 bounded rationality, not optimality; A2 few task-independent modules; A3a/b parallel across and within modules; A4 ~50 ms cognitive cycle, sequential action selection; A5 complex behaviour from cycle sequences, no global planner module. B1 long-term memories hold symbol structures with quantitative metadata; B2 global communication via working memory; B3a/b global control by procedural memory via rule-like conditions/actions that change working memory; B4 facts in declarative memory. C1 all content learnable; C2 online, incremental, side effect of performance; C3a RL over action selection; C3b composition yields automatisation; C4 declarative learning of facts and metadata tuning; C5 complex learning built from combinations of simpler forms. D1a multiple perception modules with buffers; D1b perceptual learning; D1c attentional bottleneck; D1d top-down influence from working memory; D2a multiple motor modules; D2b motor learning.

Hyperparameters and setup. Only timing: ~50 ms per cycle (A4), ~100 ms deliberate-act level.

Ablations that matter. None.

Code / data / license. No code in the article; ACT-R, Soar and Sigma are separately released architectures (not in fetched text — verify).

Stated limitations. Acknowledged gaps: metacognition, emotion, mental imagery, direct communication and learning across modules, the semantic/episodic distinction, social cognition mechanisms; no consensus on the form of motor control; Sigma's partial disagreements.

Use in our design. This is the architecture the Notebook and Skill tier *imitate* (table in §2): working memory = the assembled context; procedural memory = Skills + playbook with utility metadata; declarative memory = Notebook facts/lessons with activation-like metadata; one deliberate act per cycle = one tool call per agent step; chunking = compiling a verified multi-step trajectory (an "impasse" resolved by the senior/thinking tier) into one reusable skill; RL over utilities = updating skill/playbook success counters from verifier outcomes; declarative learning = reflector/curator delta updates. The single-user change: keep the metadata set small and architecture-fixed (frequency, recency, utility, confidence), exactly as the model prescribes. PDF: https://scispace.com/pdf/a-standard-model-of-the-mind-toward-a-common-computational-f5mhxm3hk5.pdf (open copy; publisher page https://ojs.aaai.org/index.php/aimagazine/article/view/2744)

2. Design extraction for the module

Everything in this section is **design**, derived from the papers above; it is not a paper result unless a paper is cited inline.

2.1 Algorithms to implement (consolidated pseudocode)

(A) Cognitive-cycle agent loop with chunking (Standard Model A4/A5, C3a/C3b/C4; Thousand Brains terminal conditions). This is the reference loop for the worker/senior agent and the hook points for Notebook, Skills, gate, cascade and sleep.

```
loop cycle t:                                  # one deliberate act per cycle (A4)
  WM.perceive(tool_outputs, attention_budget)  # D1c: attentional bottleneck = context budget
  WM.retrieve(Notebook, cue=WM.goal)           # B4/C4: cue-based declarative retrieval
  candidates = Skills.match(WM)                # B3a: rules whose conditions match WM
  novelty = gate.score(WM, candidates)         # design: gate = selection metadata
  if candidates and novelty < tau_fast:
      act = argmax_{c in candidates} c.utility  # C3a: utility-based selection
      tier = worker
  else:                                        # impasse (no rule / low confidence)
      tier = senior_or_thinking                # subgoal: deliberate search
      act = tier.plan(WM)                      # sequence of primitive acts
  result = motor.apply(act)                    # D2: tool call (edit / run / git)
  verdict = cascade.verify(result)             # design: cheapest-first verifiers
  Skills.update_utility(act, verdict)          # C3a: reinforcement over selection
  Notebook.delta(reflector(WM, result, verdict))# C4: declarative learning, online
  if tier != worker and verdict.pass:
      Skills.chunk(trace=tier.trace, test=verdict)   # C3b: compile impasse resolution
  if terminal(verdict, t): break               # match / no-match / time-out (TBP 10.3)
sleep (weekly): LoRA on verified traces; consolidate Notebook; retire low-utility skills (C5; Lake 4.3.2 "caching plans into
model-free values")
```

(B) Evidence-based hypothesis update with voting (Thousand Brains 9.10–10.2; formalised in 2.2). Use for keeping several candidate plans/skills alive and committing only when one dominates.

```
init: for each object o, each stored location l, each rotation r in R(l):  ev[o,l,r] = 0
step: for each hypothesis h = (o,l,r):
        loc' = l + rotate(displacement, r)
        nn = nearest_nodes(graph[o], loc', radius)
        if nn empty: delta = -1
        else: delta = pose_term(nn.best, sensed) + feature_term(nn.best, sensed)   # in [-1, 2]
        ev[h] += delta; h.location = loc'
      best = argmax ev; possible = {h : ev[h] > max_ev - p * max_ev and ev[h] > 0}
      if all ev <= 0: create new model
      vote_out = rescale(ev restricted to possible, to [-1,1]); send laterally (only if input received)
      for incoming votes v: ev[h] += distance_weighted_mean(v within radius of h)   # in [-1,1]
      terminal if |possible| == 1 (match), no positive evidence (no-match), or t >= max_steps
```

(C) JEPA training step (I-JEPA Sec. 3; V-JEPA 2 Eq. 1). Reference for the P6 representation learner over code/traces.

```
for batch x:
  targets   = sample M=4 blocks, scale (0.15,0.2), aspect (0.75,1.5)
  context   = sample 1 block, scale (0.85,1.0), aspect 1.0, minus target patches
  s_x       = f_theta(context patches)
  s_y       = stopgrad(f_thetabar(all patches))[target patches]
  loss      = (1/M) * sum_i sum_{j in B_i} || g_phi(s_x, masktok(pos_j)) - s_y[j] ||^2   # I-JEPA L2; V-JEPA 2 uses L1
  theta, phi <- AdamW(loss)
  thetabar  <- m * thetabar + (1-m) * theta,  m: 0.996 -> 1.0 linearly (I-JEPA); 0.99925 (V-JEPA 2 config)
```

(D) Action-conditioned post-training and CEM planning (V-JEPA 2 Eq. 2–5).

```
post-train (encoder frozen): for clip (x_1..x_16, s_1..s_16, a_1..a_15):
  z_k = E(x_k)
  L = (1/15) sum_k || P_phi((a_t,s_t,z_t)_{t<=k}) - z_{k+1} ||_1          # teacher forcing
    + || rollout(P_phi, a_{1:2}, s_1, z_1) - z_3 ||_1                     # 2-step rollout
plan (MPC): z_g = E(goal_image); mu, sigma = 0, 1
  repeat 10 iterations: sample 800 action sequences ~ N(mu, sigma); energy = ||P(a; s_k, z_k) - z_g||_1
                        mu, sigma <- stats of top-k;  return mu
  execute first action; replan.  Sub-goals: grasp (4 steps), place (10), final (4)
```

(E) Alberta-style continual learner skeleton (Steps 1–2, 8). For Notebook/Skill admission and retirement: per-feature step sizes (IDBD/Autostep class), generate-and-test feature replacement under a fixed budget, ranking by utility, feedback from the model to the ranking.

```
every step: state <- perceive(state, obs); pred <- model(state, action); err <- target - pred
            per-entry step size alpha_i <- meta-update(alpha_i, err)      # Step 1
            rank entries by utility; drop bottom-k; generate k candidates; test   # Step 2 / 11 (Oak)
```

(F) BabyLM-style budgeted evaluation (Sec. 3–5 + Findings). Fix the token budget (10M / 100M), forbid pretrained leakage in STRICT, score with a weighted suite (0.5 minimal-pairs, 0.3 task fine-tuning, 0.2 generalisation-vs-heuristic), report baselines and skylines.

2.2 Equations and quantities the implementation must compute

- **Evidence update (design formalisation of Thousand Brains prose):** with α_n the angle between sensed and stored point normals, α_c the angle between curvature directions, and d_f the normalised feature difference with tolerance τ_f:

$$\Delta_{\text{pose}} = \begin{cases} -1 & \text{no node within radius} \\ 1 - \frac{2\alpha_n}{180^\circ} & \text{otherwise (curvature term saturates at } 90^\circ) \end{cases}, \qquad \Delta_{\text{feat}} = \max\left(0,\, 1 - \frac{d_f}{\tau_f}\right)$$

$$\text{ev}_h \leftarrow \text{ev}_h + \Delta_{\text{pose}} + \Delta_{\text{feat}} \in [-1, 2]; \qquad \text{possible}(h) \Leftrightarrow \text{ev}_h > (1-p) \max_{h'} \text{ev}_{h'} \wedge \text{ev}_h > 0,\ p = 0.2$$

  Vote: $v_h = 2\frac{\text{ev}_h - \min}{\max - \min} - 1 \in [-1, 1]$; receiver adds $\sum_k w_k v_k / \sum_k w_k$ with w_k inverse-distance within the radius.
- **I-JEPA loss** (Sec. 3): $\mathcal{L} = \frac{1}{M}\sum_{i=1}^{M} \sum_{j \in B_i} \| \hat{s}_{y_j}(i) - s_{y_j} \|_2^2$, $M = 4$.
- **EMA target:** $\bar{\theta} \leftarrow m\bar{\theta} + (1-m)\,\theta$ (standard form; the papers state EMA without writing it), m from $0.996 \to 1.0$ linearly (I-JEPA) or 0.99925 (V-JEPA 2 config).
- **V-JEPA 2 objectives** Eq. 1–5 as transcribed in §1.3: L1 masked prediction; teacher-forcing mean over $T = 15$; rollout $T = 2$; energy $\| P(\hat{a}_{1:T}; s_k, z_k) - z_g \|_1$.
- **BabyLM aggregate (Findings, secondary):** $\text{score} = 0.5 \cdot \text{BLiMP}^{+} + 0.3 \cdot \text{GLUE} + 0.2 \cdot \text{MSGS}$ where BLiMP^{+} combines BLiMP and its supplement; MSGS is a Matthews correlation in $[-1, 1]$.
- **Standard Model timing:** one deliberate act per ~50 ms cycle; ~100 ms deliberate-act level. For our system the analogue is one tool call per agent step; the serial bottleneck is intentional.
- **Alberta signals:** experience = three time series (observation, action, scalar reward); every component consumes the *state* = recursive summary of history.

2.3 Data structures and file formats (as TypeScript-style type sketches)

```
        // Thousand Brains — Cortical Messaging Protocol (Sec. 3.1, 8)
        type CMPMessage = {
          location: [number, number, number];
          morphology: [[number,number,number],[number,number,number],[number,number,number]]; // 3x3 orthonormal: normal + 2
curvature dirs
          features?: Record<string, number | number[]>;   // colour, curvature magnitude, ... (optional)
          confidence: number;                               // [0,1]
          use: boolean;
          senderId: string;
          senderType: "sensor" | "lm" | "goal" | "vote";
        };
        type Hypothesis = { objectId: string; location: [number,number,number]; rotation: number[][]; evidence: number };
        type ObjectGraphNode = { pose: CMPMessage["morphology"]; location: [number,number,number]; features: Record<string,
unknown> };
        type ObjectGraph = { objectId: string; nodes: ObjectGraphNode[]; edges: { a: number; b: number; displacement:
[number,number,number] }[] };
        type LMState = { buffer: CMPMessage[] /* episode-scoped */; graphMemory: Record<string, ObjectGraph>; hypotheses:
Hypothesis[] };

        // JEPA configs (I-JEPA in1k_vith14_ep300.yaml; V-JEPA 2 pretrain-256px-16f.yaml)
        type JepaMaskConfig = { aspectRatio: [number, number]; encMaskScale: [number, number]; predMaskScale: [number, number];
          numEncMasks: number; numPredMasks: number; minKeep: number; allowOverlap: boolean; patchSize: number };
        type JepaTrainConfig = { epochs: number; lr: number; startLr: number; finalLr: number; warmup: number;
          weightDecay: number; finalWeightDecay: number; ema: [number, number]; predDepth: number; predEmbDim: number; lossExp:
1 | 2 };
        type ActionConditionedSample = { frames: Float32Array[] /* z_k = E(x_k), 16x16x1408 */; states: number[7][]; actions:
number[7][] };

        // Standard Model — what the Notebook / Skills imitate (design)
        type Metadata = { frequency: number; recency: number; utility: number; activation: number; confidence: number }; //
fixed set (B1)
        type DeclarativeChunk = { id: string; relation: string; args: string[]; meta: Metadata };            // Notebook fact /
lesson
        type ProductionRule  = { id: string; conditions: string[]; actions: string[]; meta: Metadata; compiledFrom?: string[]
}; // Skill / playbook entry; compiledFrom = chunking trace
        type WorkingMemory   = { goal: string; intermediate: unknown[]; buffers: { retrieval: DeclarativeChunk[]; perception:
unknown[]; motor: unknown[] } };
        type CognitiveCycleLog = { t: number; matched: string[]; selected: string; tier: "worker" | "senior" | "thinking";
verdict: "pass" | "fail" | "timeout" };

        // Alberta base agent (Fig. 2)
        type BaseAgent<S> = { perceive: (s: S, o: unknown, a: unknown, r: number) => S; policies: ((s: S) => unknown)[];
          values: ((s: S) => number)[]; model?: (s: S, a: unknown) => { sNext: S; r: number } }; // model is the missing
component today

        // BabyLM-style evaluation record
        type BudgetedEvalRun = { track: "strict" | "strict-small" | "loose"; wordBudget: 1e7 | 1e8; suite: Record<string,
number>;
          weights: { minimalPairs: 0.5; tasks: 0.3; generalisation: 0.2 }; aggregate: number; baseline: string };
```

2.4 Hyperparameters and defaults to start from (with the paper each comes from)

Quantity	Default	Source
Possible-match band p	0.20	Thousand Brains §10.1 (example value)
Evidence ranges	feature $[0,1]$, pose $[-1,1]$, step $[-1,2]$, votes $[-1,1]$	Thousand Brains §9.11–10.2
Hypothesis init	all objects x all stored locations x 2 rotations	Thousand Brains §9.10
Target blocks M; target scale; aspect	4; (0.15, 0.2); (0.75, 1.5)	I-JEPA §3, config
Context block scale; aspect	(0.85, 1.0); 1.0	I-JEPA §3
Predictor depth / width	6 (ViT-B) or 12 (ViT-L/H) / 384	I-JEPA App. A, ablations (384 > 1024)
AdamW lr / warmup / wd / EMA	1e-3 peak, cosine to 1e-6; wd 0.04 → 0.4; EMA 0.996 → 1.0	I-JEPA App. A
V-JEPA 2 masks	8 blocks @ spatial 0.15 + 2 blocks @ 0.7, temporal 1.0	vjepa2 config
V-JEPA 2 schedule	12K warmup / 228K constant / 12K cooldown (252K it)	V-JEPA 2 §2.2
V-JEPA 2-AC predictor	~300M, 24 layers, 16 heads, dim 1024, block-causal	V-JEPA 2 §3.1
Teacher-forcing / rollout horizons	$T=15$ / $T=2$	V-JEPA 2 Eq. 2–3
CEM	800 samples, 10 iterations, horizon 1, Gaussian init (0, 1)	V-JEPA 2 Table 3
Post-training data	~62 h, 16 frames @ 4 fps, 256 px	V-JEPA 2 §3.1

Quantity	Default	Source
Cognitive cycle	one deliberate act per cycle (~50 ms human)	Standard Model A4
Metadata set	frequency, recency, co-occurrence, similarity, utility, activation	Standard Model B1
Learning budget	10M (strict-small) / 100M (strict) words	BabyLM §3
Score weights	0.5 / 0.3 / 0.2	BabyLM Findings (secondary)
Planning horizon of the plan	5–10 years; 12 steps	Alberta Plan

2.5 Evaluation: how to know the module works (metrics, from the papers)

- **Recognition under movement and noise** (Thousand Brains): object + pose accuracy over YCB-style suites with random rotations and sensor noise; mean rotation error in degrees; number of steps to terminal state; first-epoch vs later-epoch accuracy for unsupervised learning (docs report 40–80% → 80–98.89%).
- **Representation quality** (I-JEPA): frozen-encoder linear probe (average-pooled features) and 1% low-shot accuracy; transfer to out-of-domain probes; local tasks (counting, depth). For code: frozen-encoder probes on span-level properties.
- **Prediction and planning** (V-JEPA 2): frozen-encoder attentive-probe accuracy; anticipation recall@5 at 1 s; zero-shot task success over 10 trials per task with image goals; planning time per action (16 s target on one consumer GPU).
- **Sample efficiency** (Lake et al.; BabyLM): hours/words of experience to reach a fixed score (DQN 924 h vs human ~2 h on Frostbite; 10M/100M word budgets); aggregate = 0.5 minimal-pairs + 0.3 tasks + 0.2 generalisation; always report a from-scratch baseline (OPT-125M-class) and a skyline (full-data model).
- **Architecture conformance** (Standard Model): every learning event logs which mechanism it used (composition/chunking, utility RL, declarative); learning is online and incremental (C2) — a module that only learns in sleep fails C2 and should be marked as such.
- **Continual learning** (Alberta): no special training phase; per-step learning and planning; feature replacement under a fixed budget without catastrophic loss of ranked features.

2.6 Known pitfalls and failure modes (from the papers' limitations)

- Pixel/token-space prediction targets degrade semantic representations (I-JEPA: 40.7 vs 66.9); small or too-large target blocks and large context both collapse quality (I-JEPA ablations) — keep targets span-level and context sparse.
- Larger patches hurt local/low-level tasks (I-JEPA ViT-G/16).
- Autoregressive rollouts accumulate error; planning search grows exponentially with horizon; goals as images only (V-JEPA 2 §4.3) — use sub-goals and replan after one action.
- Action coordinate frames inferred from uncalibrated inputs are ill-defined (V-JEPA 2): define the "proprioceptive" state of the repo explicitly.
- Evidence-based LMs need shared reference frames for votes, else a learned transform (Thousand Brains §10.2); features cannot subtract evidence ("only halfway achieved").
- Monty has no physics, fixed scale, no hierarchy yet; compositional benchmarks are at 2–40% (docs).
- Standard Model lacks metacognition, emotion, imagery, cross-module learning, semantic/episodic split, social cognition; the single-act serial bottleneck is by design.
- Alberta: perception/state learning is the least understood component; later steps are expected to change.
- Lake et al.: humans start with far richer priors; architecture search is not algorithmic; language is deferred.
- BabyLM 2023: curriculum learning mostly did not help; 10x more data barely helped at this scale; the strongest models trained for many epochs (compute-efficiency concern).

2.7 Open questions this section does not settle

- What the "object" and "reference frame" are for code (file? symbol? call graph?) so that a Thousand-Brains-style LM could store graphs with displacements.
- Whether an I-JEPA/V-JEPA objective over code embeddings yields a usable transition model at the ~300M scale a single 12 GB GPU can post-train, and what "goal embedding" ("tests pass") should be.
- How to define the scalar reward (Alberta) for a coding assistant without reintroducing a hand-labelled dataset.
- Whether Standard-Model chunking (compile a verified impasse resolution into a skill) should be done by the senior model, by the reflector, or by a program-synthesis step (cross-reference Section B).
- Numeric BPL/Omniglot results and lesion ablations from Lake et al. (not in fetched text) and the exact terminal-condition parameter names in Monty (verify in docs).
- A code analogue of BLiMP minimal pairs and MSGS for the budgeted evaluation.

3. Verified source table

arXiv id	Title	Year	Venue	Code	PDF
2412.18354	The Thousand Brains Project: A New Paradigm for Sensorimotor Intelligence	2024	arXiv (white paper)	https://github.com/thousandbrainsproject/tbp.monty	https://arxiv.org/pdf/2412.18354

arXiv id	Title	Year	Venue	Code	PDF
2301.08243	Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture	2023	CVPR 2023 (v3 Apr 2023)	https://github.com/facebookresearch/ijepa	https://arxiv.org/pdf/2301.08243
2506.09985	V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning	2025	arXiv (Meta FAIR tech report, 48 pp)	https://github.com/facebookresearch/vjepa2	https://arxiv.org/pdf/2506.09985
2208.11173	The Alberta Plan for AI Research	2022 (v3 Mar 2023)	arXiv	—	https://arxiv.org/pdf/2208.11173
1604.00289	Building Machines That Learn and Think Like People	2016 (v3 Nov 2016)	Behavioral and Brain Sciences 40, e253 (2017)	—	https://arxiv.org/pdf/1604.00289
2301.11796	Call for Papers — The BabyLM Challenge: Sample-efficient pretraining on a developmentally plausible corpus	2023	CoNLL/CMCL shared task call	https://babylm.github.io/	https://arxiv.org/pdf/2301.11796
— (no arXiv)	A Standard Model of the Mind: Toward a Common Computational Framework across Artificial Intelligence, Cognitive Science, Neuroscience, and Robotics	2017	AI Magazine 38(4), article 2744	—	https://scispace.com/pdf/a-standard-model-of-the-mind-toward-a-common-computational-f5mhxm3hk5.pdf
2504.08165 (secondary)	Findings of the BabyLM Challenge (2023)	2023/2025	CoNLL 2023	—	https://arxiv.org/pdf/2504.08165